

For a **2-year-old Doctor Appointment System**, keep the pipeline simple and interview-friendly.

Notebook 1: MongoDB → Bronze

Read data from MongoDB

```
mongo_uri = "YOUR_MONGODB_URI"
```

```
df = (  
    spark.read  
    .format("mongodb")  
    .option("spark.mongodb.read.connection.uri", mongo_uri)  
    .option("database", "Health_status_shower")  
    .option("collection", "Appointment")  
    .load()  
)
```

Save as Delta Table

```
df.write.format("delta") \  
    .mode("overwrite") \  
    .save("/FileStore/health/bronze/appointments")
```

```
display(df)
```

Notebook 2: Bronze → Silver

Clean and prepare data.

```
from pyspark.sql.functions import *
```

```
bronze_df = spark.read.format("delta").load(  
    "/FileStore/health/bronze/appointments"  
)
```

```
silver_df = (
```

```

bronze_df
.withColumn(
    "appointment_date",
    to_date(col("Date"), "yyyy-MM-dd")
)
.withColumn(
    "booking_time",
    to_timestamp(col("Created_at"))
)
)

silver_df.write.format("delta") \
    .mode("overwrite") \
    .save("/FileStore/health/silver/appointments")

display(silver_df)

```

Notebook 3: Silver → Gold Analytics

```

from pyspark.sql.functions import *

df = spark.read.format("delta").load(
    "/FileStore/health/silver/appointments"
)

1. Total Appointments Per Doctor

doctor_stats = (
    df.groupBy("Doctor_id", "Doctor_name")
        .count()
)

display(doctor_stats)

```

2. Monthly Appointments

```
monthly_stats = (  
    df.groupBy(month("appointment_date").alias("month"))  
        .count()  
)  
  
display(monthly_stats)
```

3. Repeat Patients

```
repeat_patients = (  
    df.groupBy("Patient_id")  
        .count()  
        .filter("count > 1")  
)  
  
display(repeat_patients)
```

4. Popular Time Slots

```
slot_stats = (  
    df.groupBy("Slot")  
        .count()  
        .orderBy(col("count").desc())  
)  
  
display(slot_stats)
```

Databricks Workflow

Create only **3 tasks**:

Task 1

MongoDB → Bronze

(Daily)



Task 2

Bronze → Silver



Task 3

Gold Analytics

Dependencies:

Task 2 depends on Task 1

Task 3 depends on Task 2

Project Explanation (for Internship Viva)

Bronze Layer

- Raw appointment data loaded from MongoDB.
- Stored in Delta format.

Silver Layer

- Data cleaned.
- Date and timestamp converted into proper formats.

Gold Layer

- Business reports generated:
 - Doctor-wise appointments
 - Monthly appointment trends
 - Repeat patients
 - Most booked slots

This is a simple and realistic Databricks ETL pipeline for your **Health Status Shower Doctor Appointment System** that is suitable for a 3rd-year internship project.